

Ecole

백준 문제 풀이

2025.10.24

Python

특정 도시의 거리 찾기

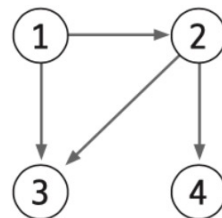
**문제[046] 특정 거리의 도시 찾기**

시간 제한 2초 | 난이도 실버 II | 백준 온라인 저지 18352번

1번부터 N번까지의 도시와 M개의 단방향 도로가 존재하고, 모든 도로의 거리는 1인 도시가 있다. 도시 X로부터 출발해 도달할 수 있는 모든 도시 중 최단 거리가 정확히 K인 모든 도시들의 번호를 출력하시오 (출발 도시 X에서 출발 도시 X로 가는 최단 거리는 항상 0이다).

예를 들어 $N = 4$, $K = 1$, $X = 1$ 일 때 다음과 같이 그래프가 구성돼 있다고 가정해 보자.

이때 1번 도시에서 출발해 도달할 수 있는 도시 중 최단 거리가 1인 도시는 2번과 3번 도시다. 4번 도시는 최단 거리가 2이므로 출력하지 않는다.

**입력**

1번째 줄에 도시의 개수(N), 도로의 개수(M), 거리 정보(K), 출발 도시의 번호(X)가 입력된다($2 \leq N \leq 300,000$, $1 \leq M \leq 1,000,000$, $1 \leq K \leq 300,000$, $1 \leq X \leq N$). 이후 M개의 줄에 걸쳐 2개의 자연수 A, B가 공백으로 구분돼 주어진다. A번 도시에서 B번 도시로 이동하는 단방향 도로가 존재한다는 뜻이다($1 \leq A, B \leq N$). 단, A와 B는 같을 수 없다.

출력

X로부터 출발해 도달 가능한 도시 중 최단 거리가 K인 모든 도시의 번호를 1줄에 1개씩 오름차순으로 출력한다. 해당하는 도시가 1개도 존재하지 않으면 -1을 출력한다.

예제 입력 1

```

4 4 2 1 // 도시 개수, 도로 개수,
거리 정보, 출발 도시 번호
1 2
1 3
2 3
2 4
  
```

예제 출력 1

```

4
  
```

예제 입력 2

```

4 3 2 1
1 2
1 3
1 4
  
```

예제 출력 2

```

-1
  
```

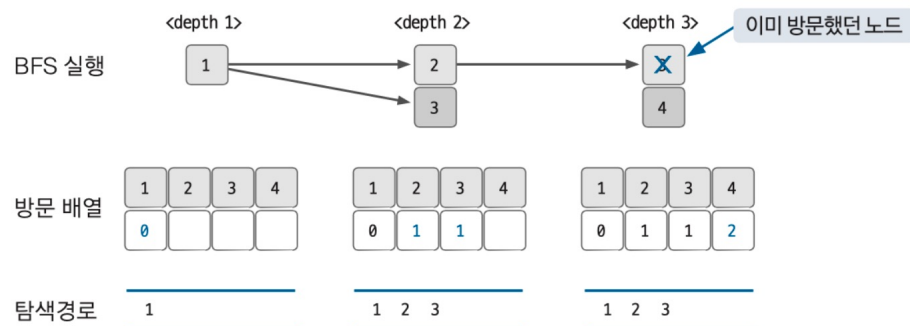


1 인접 리스트로 도시와 도로 데이터의 그래프를 구현한다.



2 BFS 탐색 알고리즘으로 탐색을 수행하면서 각 도시로 가는 최단 거릿값을 배열에 저장한다.

- 최초에는 방문 도시가 1이고, 이동하지 않았으므로 방문 배열에 0을 저장한다.
- 이후 방문하는 도시는 이전 도시의 방문 배열값 + 1을 저장하는 방식으로 이동 거리를 저장한다.



```
from collections import deque
import sys

n, m, k, s = list(map(int, sys.stdin.readline().split()))

graph = {}
for i in range(n+1):
    graph[i] = []
for i in range(m):
    a, b = list(map(int, sys.stdin.readline().split()))
    graph[a].append(b)

# bfs
dist_list = [0 for _ in range(n + 1)]
visited = [0 for _ in range(n+1)]
queue = deque([s])
# queue.append(s)
visited[s] = 1
while (queue):
    now = queue.popleft()
    for j in graph[now]:
        if visited[j] == 0:
            queue.append(j)
            visited[j] = 1
            dist_list[j] = dist_list[now] + 1

check = 0
for i in range(1, n+1):
    if dist_list[i] == k:
        print(i)
        check += 1
if check == 0:
    print(-1)
```


Java, JS

특정 도시의 거리 찾기



Java

```

public class P001 {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int n = Integer.parseInt(st.nextToken()); // 도시의 개수 (노드)
        int m = Integer.parseInt(st.nextToken()); // 도로의 개수 (간선)
        int k = Integer.parseInt(st.nextToken()); // 거리 정보
        int s = Integer.parseInt(st.nextToken()); // 출발 도시

        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int a = Integer.parseInt(st.nextToken());
            int b = Integer.parseInt(st.nextToken());
            graph.get(a).add(b);
        }

        int[] dist = new int[n + 1];
        Arrays.fill(dist, -1);

        Queue<Integer> queue = new LinkedList<>();

        queue.add(s);
        dist[s] = 0;

        while (!queue.isEmpty()) {
            int now = queue.poll();

            for (int next : graph.get(now)) {
                // 아직 방문하지 않은 노드라면
                if (dist[next] == -1) {
                    dist[next] = dist[now] + 1; // 거리 갱신
                    queue.add(next);
                }
            }
        }

        StringBuilder sb = new StringBuilder();
        boolean found = false;
        StringBuilder sb = new StringBuilder();
        boolean found = false;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == k) {
                sb.append(i).append("\n");
                found = true;
            }
        }

        if (!found) System.out.println(-1);
        else System.out.print(sb.toString());
    }
}

```

JavaScript

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const [n, m, k, s] = input[0].split(' ').map(Number);
const graph = Array.from({ length: n + 1 }, () => []);

for (let i = 1; i <= m; i++) {
    const [a, b] = input[i].split(' ').map(Number);
    graph[a].push(b);
}

const dist = new Array(n + 1).fill(-1);
const queue = [];
let head = 0; // 큐의 shift() 연산(O(N))을 대체할 포인터 (O(1))

// 6. BFS 시작 노드 설정
queue.push(s);
dist[s] = 0; // 시작 도시의 거리는 0

// 7. BFS 수행
while (head < queue.length) {
    // 큐에서 요소를 꺼냅니다. (shift() 대신 head 포인터 사용)
    const now = queue[head++];

    // 현재 도시(now)에서 갈 수 있는 다음 도시(next)들을 순회합니다.
    for (const next of graph[now]) {
        // 만약 다음 도시(next)를 아직 방문하지 않았다면
        if (dist[next] === -1) {
            // 거리를 갱신하고 (현재 도시 거리 + 1)
            dist[next] = dist[now] + 1;
            // 큐에 추가합니다.
            queue.push(next);
        }
    }
}

const answer = [];
for (let i = 1; i <= n; i++) {
    // 거리가 k인 도시를 찾습니다.
    if (dist[i] === k) {
        answer.push(i);
    }
}

if (answer.length === 0) console.log(-1);
else console.log(answer.join('\n'));

```

Python

이분 그래프 판별하기

**문제[048] 이분 그래프 판별하기**

시간 제한 2초 | 난이도 골드 IV | 백준 온라인 저지 1707번

각 집합에 속한 노드끼리 서로 인접하지 않는 두 집합으로 그래프의 노드를 나눌 수 있을 때 이 그래프를 '이분 그래프' bipartite graph 라고 한다. 그래프가 입력으로 주어졌을 때 이 그래프가 이분 그래프인지 여부를 판별하는 프로그램을 작성하시오.

↓ 입력

입력은 여러 개의 사례로 구성돼 있는데, 1번째 줄에 테스트 케이스의 개수 $K(2 \leq K \leq 5)$ 가 주어진다. 각 사례의 1번째 줄에 그래프의 노드의 개수 $V(1 \leq V \leq 20,000)$ 와 에지의 개수 $E(1 \leq E \leq 200,000)$ 가 빈칸을 사이에 두고 순서대로 주어진다. 각 노드에는 1부터 V 까지 차례로 번호가 붙어 있다. 이어서 2번째 줄부터 E 개의 줄에 걸쳐 에지와 관련된 정보가 주어지는데, 각 줄에 인접한 두 노드의 번호가 공백 문자를 사이에 두고 주어진다.

↑ 출력

K 개의 줄에 걸쳐 입력으로 주어진 그래프가 이분 그래프이면 YES, 아니면 NO를 순서대로 출력한다.

예제 입력 1

```
2      // 테스트 케이스 개수
3 2    // 노드 개수, 에지 개수
1 3
2 3
4 4
1 2
2 3
3 4
4 2
```

예제 출력 1

```
YES
NO
```



Python

이분 그래프 판별하기 - 프롭프트

내가 파이썬 코드로 백준 문제를 풀었는데 이 코드 자바 코드로 변환
해줘
1707 이분 그래프 판별하기...

생각하는 과정 표시

제출하신 파이썬 코드를 자바 코드로 변환했습니다.

로직은 거의 동일합니다.

- 빠른 입출력을 위해 `BufferedReader` 와 `BufferedWriter` 를 사용합니다.
- Python의 `global result` 변수는 Java에서 `static boolean result` 와 같이 정적(static) 필드로 선언하여 `dfs` 함수와 `main` 함수가 공유하도록 했습니다.
- Python의 `graph = [[] for _ in range(v+1)]` 는 Java에서 `ArrayList<Integer>[] graph` 를 선언하고, 각 인덱스에 `new ArrayList<>()` 를 할당하는 방식으로 구현합니다.
- Python의 `visited = [-1] * (v+1)` 는 `int[] visited` 를 선언하고 `Arrays.fill(visited, -1)` 로 초기화합니다.
- `dfs` 함수 내에서 `result` 가 `false` 가 되는 즉시 `return` 하여 불필요한 탐색을 중단하는 로직도 동일하게 구현했습니다.

백준 1707번 (이분 그래프 판별) Java 코드

Java

```
import java.io.*;
```

백준에서 `BufferedReader`를 사용해야 시간초과에 안걸리는 이유가 뭐
야?

생각하는 과정 표시

`BufferedReader` 가 `Scanner` 보다 압도적으로 빠른 이유는 **I/O (입출력) 시스템 콜(System Call)** 횟수와 **데이터 처리 방식**의 근본적인 차이 때문입니다. ⚡

백준과 같은 알고리즘 문제 풀이(CP, Competitive Programming) 환경에서는 수만 ~ 수백만 줄의 입력을 받아야 하는 경우가 많습니다. 이때 `Scanner` 를 사용하면 이 I/O 작업 자체가 병목이 되어 시간 초과가 발생할 수 있습니다.

핵심적인 차이점들을 나누어 설명해 드릴게요.

1. '버퍼(Buffer)'의 존재 유무 (가장 큰 이유)

`BufferedReader` 는 이름 그대로 **"버퍼"**를 사용합니다.

• `BufferedReader` (버퍼) 📦

- 데이터를 읽을 때, 한 번에 8KB(기본값)와 같이 큰 덩어리(버킷)로 데이터를 '미리' 퍼옵니다. (이것을 '버퍼에 채운다'고 합니다.)
- 우리가 `readLine()` 메소드를 호출하면, `BufferedReader` 는 실제 시스템(키보드/파일)에 매번 접근하는 것이 아니라, 이미 퍼 온 메모리상의 버퍼에서 데이터를 한 줄씩 쓱쓱 꺼내줍니다.

• `Scanner` (느림) 🐢

- `Scanner` 도 내부적으로 작은 버퍼(약 1KB)를 사용하긴 하지만, 작동 방식이 다릅니다.
- `nextInt()`, `next()` 등을 호출할 때마다 입력의 '토큰(단어)'을 찾기 위해 더 자주 **시스템 I/O**를 호출 가능성이 큼니다.
- 데이터를 큰 덩어리로 미리 읽어오는 것이 아니라, 요청할 때마다 필요한 만큼만 읽어오는 경향이 있습니다.

비유:

- `BufferedReader` : 우물에서 물을 8L짜리 큰 물통으로 한 번 퍼 온 뒤, 컵에 따르는 방식. (I/O 호출 1회)
- `Scanner` : 우물에서 200ml 컵으로 직접 물을 40번 퍼다 마시는 방식. (I/O 호출 40회)

우물까지 왔다 갔다 하는 시간(I/O 호출)이 물을 마시는 시간(처리)보다 훨씬 오래 걸립니다.

이번에는 자바스크립트 코드로 변경해줘

생각하는 과정 표시

네, 해당 파이썬 코드를 JavaScript(Node.js) 코드로 변환했습니다.

가장 중요한 차이점은 **재귀(Recursion)** 처리입니다.

Python에서는 `sys.setrecursionlimit` 로 재귀 깊이를 늘릴 수 있지만, Node.js(V8 엔진)는 이 한도가 훨씬 낮고 직접 늘리기가 어렵습니다. Python 코드를 그대로 재귀 DFS로 번역하면 **V (정점 개수)**가 클 때 **'Maximum call stack size exceeded' (스택 오버플로)** 오류가 발생합니다.

따라서 JavaScript 환경에서는 이 문제를 풀 때 재귀 DFS 대신 **스택(Stack)**을 이용한 **반복(Iterative) DFS** 또는 **큐(Queue)**를 이용한 **BFS**로 구현하는 것이 표준적입니다.

여기서는 원본 코드의 `dfs` 함수 의도를 살려 **스택을 이용한 DFS** 방식으로 변환했습니다.

백준 1707번 (이분 그래프 판별) JavaScript 코드

JavaScript

```
const fs = require('fs');
```

Python 코드와의 주요 차이점

- 입력 처리:** Node.js에서는 `fs.readFileSync` 를 사용해 입력을 통째로 읽어온 뒤, `split('\n')` 로 줄 바꿈 기준 분리하고, `lineIndex` 변수로 한 줄씩 처리하는 것이 일반적입니다.
- DFS 구현:** 스택 오버플로를 피하기 위해 재귀 대신 `while` 문과 `stack` 배열을 사용한 **반복(Iterative) 방식**으로 변경했습니다. 로직의 핵심(방문 안 했으면 다른 팀 할당, 방문했는데 같은 팀이면 `false`)은 동일합니다.
- 자료구조 초기화:**
 - Python: `graph = [[] for _ in range(v+1)]`
 - JS: `const graph = Array.from({ length: v + 1 }, () => [])`
 - Python: `visited = [-1] * (v+1)`
 - JS: `const visited = new Array(v + 1).fill(-1)`
- 전역 변수:** Python의 `global result` 대신, `for` 문 내에 `let isBipartite = true;` 라는 지역 변수를 선언하고 `dfs` 함수의 반환 값(`true` / `false`)을 받아 처리하는 방식으로 변경했습니다.



Python

이분 그래프 판별하기 - 프롬프트

DFS 말고 BFS로 문제를 풀었을때 어떤 차이점이 있어?

생각하는 과정 표시

BFS(너비 우선 탐색)로 이분 그래프를 판별할 때, DFS(깊이 우선 탐색)와 비교하면 탐색 순서와 사용하는 자료구조가 가장 크게 다릅니다.

결론부터 말하면, 이 문제(이분 그래프 판별)에서는 **DFS를 쓰는 BFS를 쓰는 정답을 찾는 데 아무런 차이가 없습니다**. 둘 다 시간 복잡도는 $O(V + E)$ (정점 개수 + 간선 개수)로 동일합니다.

가장 큰 차이점들을 짚어드릴게요.

1. 핵심 자료구조: 스택(Stack) vs 큐(Queue)

- DFS (깊이 우선 탐색):** **스택(Stack)**을 사용합니다. (재귀 함수를 쓰면 '호출 스택'이 암묵적으로 사용됩니다.)
 - LIFO (Last-In, First-Out) 방식입니다.
 - 가장 최근에 방문한 노드(스택의 top)에 연결된 노드부터 탐색합니다.
- BFS (너비 우선 탐색):** **큐(Queue)**를 사용합니다.
 - FIFO (First-In, First-Out) 방식입니다.
 - 가장 먼저 방문했던 노드(큐의 front)에 연결된 노드부터 순서대로 탐색합니다.

2. 탐색 순서: 깊이 vs 너비

이것이 두 알고리즘의 본질적인 차이입니다.

- DFS (깊이 우선): "한 늪만 뚫다"**
 - 시작점에서 출발해 한 방향으로 최대한 깊게 들어갑니다.
 - 막다른 길(더 이상 방문할 이웃이 없음)에 도달하면, 한 칸 뒤로 돌아와(Backtracking) 다른 길을 탐색합니다.
- BFS (너비 우선): "주변부터 쫓긴다"**
 - 시작점에서 출발해, 가까운 이웃들을 모두 먼저 방문합니다. (레벨 1)
 - 그다음, 그 이웃들의 이웃들을 모두 방문합니다. (레벨 2)
 - 마치 물에 돌을 던졌을 때 물결이 퍼져나가는 모습과 같습니다.

3. 이분 그래프 문제에서의 적용

'이분 그래프 판별 로직' (현재 노드와 이웃 노드의 색(팀)이 달라야 한다)은 동일합니다. 단지 그 로직을 적용하는 순서만 달라집니다.

- DFS 방식:**
 - A**(1팀) 방문 -> 이웃 **B** 방문
 - B**(2팀) 방문 -> 이웃 **C** 방문
 - C**(1팀) 방문 -> 이웃 **D** 방문 ...
 - 이렇게 재귀/스택으로 깊게 파고들다가, 이미 방문한 이웃을 만났는데 같은 팀이면 "NO"를 반환합니다.
- BFS 방식:**
 - A**(1팀) 를 큐에 넣습니다.
 - A** 를 큐에서 빼고, **A**의 모든 이웃 (**B**, **C**) 을 방문합니다.
 - B**(2팀), **C**(2팀) 을 큐에 넣습니다.
 - B** 를 큐에서 빼고, **B**의 모든 이웃 (**D**, **E**)을 방문합니다.
 - D**(1팀), **E**(1팀) 을 큐에 넣습니다.
 - C** 를 큐에서 빼고, **C**의 이웃 (**F**)를 방문...
 - 이렇게 레벨별로 탐색하다가, 큐에 넣으려는 이웃이 이미 방문했는데 같은 팀이면 "NO"를 반환합니다.

둘 사이의 시간복잡도 차이는 없는거야?

생각하는 과정 표시

네, 맞아요. 시간 복잡도 차이는 없습니다. 🙄

이분 그래프 판별 문제에서 DFS와 BFS는 완전히 동일한 시간 복잡도를 가집니다.

- 시간 복잡도: $O(V + E)$
 - V : 정점(Vertex)의 개수
 - E : 간선(Edge)의 개수

왜 $O(V + E)$ 로 동일할까요?

두 알고리즘 모두 "모든 정점과 모든 간선을 딱 한 번씩만 확인한다"는 공통점이 있기 때문입니다.

- V (정점)에 대한 시간:**
 - `visited` 배열을 통해 이미 방문한 정점은 다시 방문(탐색)하지 않습니다.
 - DFS:** 모든 정점을 재귀 호출 스택에 한 번씩 올라가나 내립니다.
 - BFS:** 모든 정점을 큐(Queue)에 한 번씩 넣고(enqueue) 뺍니다(dequeue).
 - 결국, 두 방식 모두 모든 정점을 정확히 한 번씩 처리합니다.
- E (간선)에 대한 시간:**
 - 그래프가 인접 리스트로 구현되어 있다고 가정할 때,
 - 특정 정점을 방문하면 그 정점에 연결된 모든 간선(이웃)을 확인합니다.
 - DFS든 BFS든** 모든 정점을 방문하는 과정에서 **모든 간선을 결국 한 번씩** (양방향 그래프에서는 각 방향으로 한 번씩, 총 두 번) 확인하게 됩니다.

파이썬 코드에서 sys.setrecursionlimit(10**6)은 백준의 최대재귀깊이 때문에 설정하는거야?

생각하는 과정 표시

네, 정확합니다! 🙌

그 코드를 추가하는 이유는 **Python의 기본 재귀 깊이 제한(Default Recursion Depth Limit)**이 백준의 채점 데이터(특히 깊은 그래프)를 처리하기에 너무 얕기 때문입니다.

- Python의 기본 설정:** Python은 무한 재귀로 인한 **스택 오버플로(Stack Overflow)**를 방지하기 위해, 재귀 호출의 최대 깊이를 기본적으로 **1,000** 정도로 제한합니다.
- 알고리즘 문제의 한계:** 하지만 백준의 1707번 문제처럼 정점(V)이 20,000개인 경우, 그래프가 만악 일자(chain) 형태로 깊게 늘어져 있다면 DFS 탐색은 1,000번을 훌쩍 넘는 재귀 호출을 할 수 있습니다.
- 문제 발생:** 이 코드가 없으면, Python의 기본 제한인 1,000을 넘는 순간 `RecursionError` (재귀 예러)가 발생하면서 코드가 런타임 예러로 종료됩니다.
- 해결책:** `sys.setrecursionlimit(10**6)` 코드는 Python 인터프리터에게 "재귀 호출 한도를 1,000,000(10의 6승)까지 늘려달라"고 요청하는 것입니다.

요약: 그 코드는 백준의 채점 환경이 요구하는 것이 아니라, 백준의 테스트 케이스를 통과하기 위해 Python의 기본 제한을 개발자가 직접 풀어주는 코드입니다. (다행히 백준의 채점 서버는 이 제한을 늘리는 것을 허용해 줍니다.)

이분 그래프가 실생활에서 적용하고 있는 예시나 활용 가능한 분야 있을까

생각하는 과정 표시

네, 이분 그래프는 ***두 개의 서로 다른 그룹 간의 관계***를 모델링하는 데 매우 강력해서 실생활 여러 분야에서 핵심적으로 활용됩니다.

알고리즘 문제(1707번)에서는 단순히 "두 그룹으로 나눌 수 있는가?"를 판별했지만, 실생활에서는 ***두 그룹을 '매칭'시키거나 '추천'***하는 문제에 널리 쓰입니다.

1. 매칭 시스템 (Matching System)

이분 그래프가 가장 고전적이고 활발하게 쓰이는 분야입니다. 한쪽 그룹(A)과 다른 쪽 그룹(B)을 짝지어주는 모든 문제에 적용됩니다.

- 구인/구직 (채용 플랫폼)**
 - 그룹 A:** 구직자 목록
 - 그룹 B:** 회사(일자리) 목록
 - 간선:** 구직자가 특정 회사에 지원했거나, 회사가 요구하는 스킬을 구직자가 보유한 경우.
 - 활용:** 가능한 한 많은 사람을 일자리에 매칭 (최대 이분 매칭)시키거나, 모든 회사가 최소 1명은 채용하게 하는 등의 최적의 조합을 찾습니다.
- 소개팅 앱 (데이팅 서비스)**
 - 그룹 A:** (이성애자 기준) 남성 회원
 - 그룹 B:** 여성 회원
 - 간선:** 서로 '좋아요'를 눌렀거나, 매칭 조건(거리, 나이 등)이 맞는 경우.
 - 활용:** 동시에 가장 많은 커플(매칭)을 성사시키는 방법을 계산합니다.
- 학교/병원 (매칭 문제)**
 - 그룹 A:** 학생 (또는 신입 레지던트)
 - 그룹 B:** 수강할 과목 (또는 배치될 병원 과)
 - 간선:** 학생이 1~3지방 등으로 선호도를 표시한 경우.
 - 활용:** 학생들의 선호도를 최대한 만족시키면서 모든 과목(병원 과)에 인원을 배정하는 '안정적인 매칭'을 찾는 데 쓰입니다. (e.g., **게알-세콜리 알고리즘**)

저바코드는 `ArrayList<Integer>[]`를 사용해 인접 리스트를 구현했는데 Python은 `[] for _ in range(v+1)`을 사용했는데 이 두 언어의 구현 방식이 메모리, 성능 면에서 어떤 차이가 있어?

생각하는 과정 표시

매우 날카로운 질문입니다! 🧐
결론부터 말씀드리면, 알고리즘의 시간 복잡도(O)는 동일하지만, 실제 실행 속도는 **Java가 훨씬 빠르고 메모리 사용량도 Java가 더 적습니다.**

두 언어의 구현 방식은 겉보기엔 비슷하지만, 내부적으로는 근본적인 차이가 있습니다.

핵심 비유: '고정된 파일 캐비닛 vs 늘어나는 아코디언 폴더'

- Java:** `ArrayList<Integer>[] graph`
 - `new ArrayList<Integer>[v+1]`는 "칸의 개수가 **v+1**개로 고정된 파일 캐비닛"을 만들어 주는 것과 같습니다.
 - 각 `graph[i]`에는 `ArrayList`라는 "내용물에 따라 늘어나는 아코디언 폴더"를 하나씩 넣습니다.
 - 특징: 캐비닛 자체의 크기(정점의 개수)는 절대 변하지 않습니다.
- Python:** `graph = [[] for _ in range(v+1)]`
 - 이것은 "**v+1**개의 슬롯을 가진 아코디언 폴더"를 만드는 것과 같습니다.
 - 그리고 그 각 슬롯에 "또 다른 아코디언 폴더"를 하나씩 끼워 넣습니다.
 - 특징: 바깥쪽 폴더(`graph`)조차도 이론상으로는 `graph.append()`로 크기를 늘릴 수 있습니다. (물론 이 문제에선 안 쓰지만)

1. 성능 (속도) 차이

알고리즘 복잡도: $O(V + E)$ (동일)

DFS/BFS 알고리즘을 수행하는 관점에서는 관점에서는 이론적인 시간 복잡도는 **$O(V+E)$ 로 같습니다.**

- `graph[i]` (특정 정점의 이웃 리스트 접근): 두 언어 모두 $O(1)$ 입니다.
- `graph[i].add()` / `graph[i].append()` (간선 추가): 두 언어 모두 "분할 실행(Amortized) $O(1)$ "입니다. (작은 배열 크기를 가끔 2배로 늘리는 비용 포함)

실제 실행 속도: **Java > Python (Java 8)** 🚀

이론이 아닌 실제 속도는 Java가 훨씬 빠릅니다.

- 컴파일 vs 인터프리터:** Java는 JIT(Just-In-Time) 컴파일러를 통해 코드를 실행 시점에 고도로 최적화된 기계어로 번역합니다. Python(Pythoon)은 인터프리터 언어로, 한 줄씩 번역하며 실행하므로 각 작업(Operation)마다 오버헤드가 발생합니다.
- 정적 타입 vs 동적 타입:** Java는 `ArrayList<Integer>` 러고 타입을 명확히 알아서 "이전 정수 리스트"라고 인지하고 최적화합니다. Python은 `graph[i]`에 접근할 때마다 "이게 리스트가 하나? `append` 라는 메서드가 있나?" 등을 매번 확인(Dynamic Dispatch)해야 하므로 느립니다.

2. 메모리 차이

Java < Python (Java 8) 📦

메모리 사용량은 Python이 Java보다 훨씬 더 많이 사용합니다.

1. 자료구조의 기본 구조:

- Java:** `ArrayList[]`는 "인덱스 배열(Array)"입니다. 메모리상에 `ArrayList` 객체를 가리키는 주소값들이 $v+1$ 개 연속으로 놓입니다.
- Python:** `list`는 "동적 배열(Dynamic Array)"입니다. 다른 `list` 객체를 가리키는 주소값들이 들어있는 `list` 객체입니다.

2. 데이터(정수) 저장 방식 (가장 큰 차이):

- Java:** `int`를 `ArrayList<Integer>`에 넣으면 '박싱(Boxing)'이 일어나 `Integer` 객체가 됩니다. 이 객체는 [객체 헤더 + 4바이트(int 32)] 정도의 메모리를 씁니다.
- Python:** Python의 `int`는 단순한 숫자가 아니라 "대체한 '객체(Object)'"입니다. [객체 헤더 + 참조 카운트(GC용) + 타입 정보 + 실제 숫자 값(십진기어 기반 길이)] 등 Java의 `Integer` 객체보다 훨씬 더 많은 기본 오버헤드를 가집니다.

결론: Python은 리스트 자체의 오버헤드도 조금 더 크고, 그 안에 저장되는 데이터(숫자) 하나하나의 오버헤드가 Java보다 훨씬 크기 때문에 총 메모리 사용량이 더 많습니다.

실제 현업에서도 저 알고리즘을 그대로 쓰지않고 개념만 사용하면서 저 코드를 기반으로 하는거야 아니면 라이브러리같은게 따로 있어?

생각하는 과정 표시

정말 좋은 질문입니다. 그 둘을 정확히 구분하고 제시해요.

결론부터 말씀드리면, **개념을 사용해 어떤 라이브러리를 써야 할지 결정하고, 실제 구현은 대부분 잘 만들어진 라이브러리에 맡깁니다.**

방금 작성하신 백준 코드는 '날것' 그대로의 구현(raw implementation)이고, 현업에서는 이것을 이미 고도로 최적화해 둔 '도구'를 가져다 씁니다.

1. 대부분의 현업: 잘 만들어진 "라이브러리"를 씁니다 🚀

일반적인 웹/앱 개발(어플리케이션 개발)에서는 시간과 안정성이 생명이죠. 우리가 `ArrayList`를 직접 만들지 않고 Java가 제공하는 것을 쓰듯이, 그래프 알고리즘도 이미 검증된 라이브러리를 사용합니다.

- 예시 (Python의 `networkx` 라이브러리):**
 - 170만 문제(이론 그래프 답변)를 현업에서 `networkx`로 풀다면 코드는 이제 정품입니다.

Python

```
import networkx as nx

# 1. 빈 그래프(G) 생성
G = nx.Graph()
```

```
# 2. 간선 추가 (v, w = 입력받았고 가정)
for _ in range(E):
    start, end = map(int, input().split())
    G.add_edge(start, end) # 여기서 인접 리스트처럼 관리함

# 3. 비어 있는 그래프 판별
if nx.is_bipartite(G): # 예, (내부적으로 BFS나 DFS가 돌아감)
    print("YES")
else:
    print("NO")
```

예시 (최단 경로 찾기):

- 우리가 BFS를 직접 `Queue`로 구현할 필요 없이, 라이브러리의 함수 한 줄이면 됩니다.
- `path = nx.shortest_path(G, source=1, target=5)`
- 이 `shortest_path` 함수는 내부적으로 **BFS 알고리즘** 개념을 기반으로 동작합니다.

왜 라이브러리를 쓸까요?

- 생산성:** `nx.is_bipartite(G)` 한 줄이면 되는데, 20~30줄짜리 DFS/BFS 코드를 또 할 필요가 없습니다.
- 안정성:** 수많은 개발자가 사용하고 검증(테스트)받기 때문에 버그가 거의 없습니다.
- 최적화:** C/C++ Python으로 작성된 코어 로직을 사용하는 경우가 많아, 우리가 순수 Python/Java로 쓴 것보다 훨씬 빠릅니다.

2. 그럼에도 "개념"과 "기본 코드"를 배우는 이유 🧠

그럼 "이차원 라이브러리"의 쓸 거, 백준에서 왜 이렇게 물어보는지의 의의가 생길 수 있습니다.

그 개념을 모르면, 수많은 라이브러리 함수 중에 무엇을 써야 할지 모르기 때문입니다.

1. 올바른 도구 선택:

- "A에서 B로 가는 최단 경로가 필요해" -> (머릿속: "가장치는 최단 경로? BFS") -> `nx.shortest_path` 함수를 검색해서 씁니다.
- "A에서 B로 가는 경로가 있는지 아닌지 확인" -> (머릿속: "아무거나. DFS가 편하겠다.") -> `nx.has_path` 함수를 씁니다.
- "사용자와 상품을 매칭시키고 싶어" -> (머릿속: "두 그룹 간의 매칭? 이분 그래프 매칭") -> `nx.bipartite.maximum_matching` 함수를 씁니다.

2. 성능 예측 (시간 복잡도):

- "지금 쓸 데이터(V)가 100만 개인데, 이 라이브러리의 함수의 시간 복잡도가 $O(V^2)$ 이냐?" 쓰면 서비스 터지겠다."
- "아, 저런 $O(V+E)$ 도 나쁘게 편찮아."
- 이처럼 `sys.setrecursionlimit`이나 `BufferedReader`를 고민했던 경험이, 현업에서 **병목 현상**을 예측하고 방지하는 핵심 능력이 됩니다.

3. 응용 및 디버깅:

- 라이브러리가 내가 원하는 기능을 100% 제공하지 않을 때가 있습니다.
- "DFS 기반으로 탐색하는데, 특정 조건을 만족하면 그 경로는 더 이상 탐색하지 말아야지."
- 이런 '커스터마이징'은 **DFS/BFS의 원본 코드(기본 개념)**를 모르면 절대 불가능합니다.

알고리즘 문제 풀이에서는 `sys.stdin.readline`을 쓰는 게 거의 표준인데, 만약 Python의 일반적인 `input()` 함수를 사용했다면 입출력에서 얼마나 큰 성능 차이가 발생할까?

생각하는 과정 표시

엄청난 차이가 납니다. 🧐

문제의 입력 크기(N)에 따라 적게는 수십 배에서 많게는 **100배 이상**까지 성능 차이가 발생할 수 있습니다.

Java에서 `Scanner` 대신 `BufferedReader`를 쓰는 이유와 거의 같습니다. 100만 줄의 입력을 받는다고 가정할 때, `input()`을 사용하면 알고리즘은 **0.1초 만에 풀어도 입력만 받다가 시간 초과**가 날 수 있습니다.

`input()` VS `sys.stdin.readline()`

두 함수의 가장 큰 차이는 **"작업의 오버헤드(Overhead)"**입니다.

1. `input()`: 친절하지만 무거운 '원재료' 📦

`input()` 함수는 사용하기 편리하도록 여러 가지 부가 작업을 수행하는 '고수준(high-level)' 함수입니다.

`n = input()` 한 줄은 내부적으로 다음과 같이 동작합니다.

- `(stdout 버퍼를 flush 모르니 비용입니다.)`
- `sys.stdin.readline()`을 호출해서 한 줄을 읽어옵니다. (e.g., `"123\n"`)
- 읽어온 문자열의 끝에 붙은 개행 문자(`\n`)를 제거합니다. (e.g., `"123"`)
- 그 결과를 반환합니다.

여기서 가장 큰 병목은 ****3번(개행 문자 제거)****입니다.

2. `sys.stdin.readline()`: 무뎌보이지만 빠른 '원재료' 🚀

`sys.stdin.readline()`은 버퍼에서 데이터를 읽어오는 '저수준(low-level)' I/O 메소드입니다.

`line = sys.stdin.readline()` 한 줄은 딱 한 가지 일만 합니다.

- I/O 버퍼에서 `\n` (개행 문자)를 만날 때까지의 바이트를 읽습니다.
- `\n`을 포함한 문자열 (e.g., `"123\n"`)을 그대로 반환합니다.

간단한 테스트만 해봐도 100만 줄의 정수를 읽는데 `input()`은 ****몇 초(2~5초)****가 걸리는 반면,

`sys.stdin.readline()`은 **0.5초 이내**에 끝납니다.

결론: 알고리즘 문제 풀이(PS) 환경에서 2초의 시간제한이 주어질 때, `input()`을 쓰면 입력만 받다가 2초를 다 써 버려서 '시간 초과'를 받게 됩니다. `sys.stdin.readline()`은 그 시간을 0.5초로 줄여서, 남은 1.5초를 알고리즘 연산에 쓸 수 있게 해줍니다.



- 1 입력된 그래프 데이터를 인접 리스트로 구현한다.

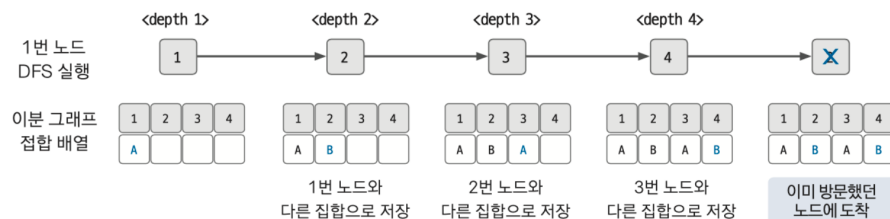


- 2 모든 노드로 각각 DFS 탐색 알고리즘을 적용하 탐색을 수행한다.

현재 노드에서 연결된 노드 중 이미 방문한 노드가 나와 같은 집합이면 이분 그래프가 아니다.

이분 그래프가 아니면 이후 노드는 탐색하지 않는다.

▶ 2개 집합을 각각 A, B로 표현했습니다.



- 3 이분 그래프 여부를 정답으로 출력한다.

출발 노드(4)와 도착 노드(2)의 집합이 같으므로 이분 그래프 불가능 $\Rightarrow \therefore$ NO 출력

- 4 사례의 개수만큼 과정 1~3을 반복한다.

Python

```
import sys
sys.setrecursionlimit(10**6)
t = int(sys.stdin.readline())

def dfs(node):
    global result

    for neighbor in graph[node]:
        if (visited[neighbor] == -1):
            if (visited[node] == 1):
                visited[neighbor] = 2
            elif (visited[node] == 2):
                visited[neighbor] = 1

            dfs(neighbor)
        else:
            if (visited[node] == visited[neighbor]):
                result = False
                return

    result = True
    return

for _ in range(t):
    v, e = map(int, sys.stdin.readline().split())

    visited = [-1] * (v+1)

    graph = [[] for _ in range(v+1)]

    for _ in range(e):
        start, end = map(int, sys.stdin.readline().split())
        graph[start].append(end)
        graph[end].append(start)

    result = True

    for i in range(1, v+1):
        if (visited[i] == -1):
            visited[i] = 1
            dfs(i)
            if (result == False):
                break

    if (result == False):
        print("NO")
    else:
        print("YES")
```


Java, JS

이분 그래프 판별하기



Java

```
public class P002 {
    static ArrayList<ArrayList<Integer>> graph;
    static int[] visited; // -1: 미방문, 1: 1번 그룹, 2: 2번 그룹
    static boolean isBipartite;

    static void dfs(int node) {
        if (!isBipartite) {
            return;
        }
        for (int neighbor : graph.get(node)) {
            if (visited[neighbor] == -1) {
                visited[neighbor] = (visited[node] == 1) ? 2 : 1;
                dfs(neighbor);
            }
            else if (visited[node] == visited[neighbor]) {
                isBipartite = false;
                return;
            }
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;
        StringBuilder sb = new StringBuilder();

        int t = Integer.parseInt(br.readLine()); // 테스트 케이스 수

        for (int i = 0; i < t; i++) {
            st = new StringTokenizer(br.readLine());
            int v = Integer.parseInt(st.nextToken()); // 정점(Vertex) 수
            int e = Integer.parseInt(st.nextToken()); // 간선(Edge) 수

            // 1. 그래프 및 방문 배열 초기화
            graph = new ArrayList<>();
            for (int j = 0; j <= v; j++) {
                graph.add(new ArrayList<>());
            }

            // -1 (미방문) 상태로 초기화
            visited = new int[v + 1];
            Arrays.fill(visited, -1);
        }
    }
}
```

```
// 2. 간선 정보 입력
for (int j = 0; j < e; j++) {
    st = new StringTokenizer(br.readLine());
    int start = Integer.parseInt(st.nextToken());
    int end = Integer.parseInt(st.nextToken());
    // 양방향 그래프
    graph.get(start).add(end);
    graph.get(end).add(start);
}

// 3. 이분 그래프 판별
isBipartite = true; // 일단 true로 가정

for (int j = 1; j <= v; j++) {
    // 아직 방문하지 않은 정점(새 컴포넌트의 시작점)이라면
    if (visited[j] == -1) {
        visited[j] = 1; // 1번 그룹(색)으로 칠하고 시작
        dfs(j);
    }
    // DFS 도중 이분 그래프가 아님이 판명되면 즉시 중단
    if (!isBipartite) {
        break;
    }
}

// 4. 결과 출력
sb.append(isBipartite ? "YES" : "NO").append("\n");
System.out.print(sb.toString());
}
```



JS

이분 그래프 판별하기 - 정답

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

let line = 0; // 현재 읽고 있는 줄을 추적
const t = Number(input[line++]); // 테스트 케이스 수
const answer = []; // 출력을 모을 배열

// 2. DFS 함수
// (변수들을 파라미터로 넘겨 재귀 스택 오버플로우를 조금이나마 방지)
function dfs(node, graph, visited) {
    // 현재 노드에 연결된 모든 이웃 탐색
    for (const neighbor of graph[node]) {
        // 1. 이웃이 미방문 상태라면
        if (visited[neighbor] === -1) {
            // 현재 노드와 반대되는 색(1 <-> 2)으로 칠함
            visited[neighbor] = visited[node] === 1 ? 2 : 1;
            // 재귀 호출. 만약 false(모순)가 반환되면 즉시 전파
            if (!dfs(neighbor, graph, visited)) {
                return false;
            }
        }
        // 2. 이웃이 방문되었는데, 현재 노드와 색이 같다면
        else if (visited[node] === visited[neighbor]) {
            // 모순 발생!
            return false;
        }
    }
    // 이 노드에서 모순이 없었으면 true 반환
    return true;
}
```

```
// 3. 테스트 케이스 반복
for (let i = 0; i < t; i++) {
    const [v, e] = input[line++].split(' ').map(Number);
    const graph = Array.from({ length: v + 1 }, () => []);
    const visited = new Array(v + 1).fill(-1); // -1: 미방문, 1, 2: 그룹(색)

    for (let j = 0; j < e; j++) {
        const [start, end] = input[line++].split(' ').map(Number);
        graph[start].push(end);
        graph[end].push(start);
    }

    let isBipartite = true;

    // 4. 모든 정점(컴포넌트) 순회
    for (let j = 1; j <= v; j++) {
        // 아직 방문하지 않은 정점(새 컴포넌트) 발견
        if (visited[j] === -1) {
            visited[j] = 1; // 1번 그룹(색)으로 시작
            if (!dfs(j, graph, visited)) {
                isBipartite = false;
                break; // 모순이 발견되면 즉시 중단
            }
        }
    }
    answer.push(isBipartite ? "YES" : "NO");
}

// 5. 결과 출력
console.log(answer.join('\n'));
```

Python

집합 표현하기



빈출

문제[050] 집합 표현하기

시간 제한 2초 | 난이도 골드 IV | 백준 온라인 저지 1717번

초기에 $\{0\}, \{1\}, \{2\}, \dots, \{n\}$ 이 각각 $n+1$ 개의 집합을 이루고 있다. 여기에 합집합 연산과 두 원소가 같은 집합에 포함돼 있는지를 확인하는 연산을 수행하려고 한다. 집합을 표현하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 $n(1 \leq n \leq 1,000,000)$, $m(1 \leq m \leq 100,000)$ 이 주어진다. m 은 입력으로 주어지는 연산의 개수다. 다음 m 개의 줄에는 각각의 연산이 주어진다. 합집합은 $0 \ a \ b$ 의 형태로 입력이 주어진다. 이는 a 가 포함돼 있는 집합과 b 가 포함돼 있는 집합을 합친다는 의미다. 두 원소가 같은 집합에 포함돼 있는지를 확인하는 연산은 $1 \ a \ b$ 의 형태로 입력이 주어진다. 이는 a 와 b 가 같은 집합에 포함돼 있는지를 확인하는 연산이다. a 와 b 는 n 이하의 자연수 또는 0이고, 같을 수도 있다.

↑ 출력

1로 시작하는 입력에 1줄에 1개씩 YES 또는 NO로 결과를 출력한다.

예제 입력 1

```
7 8 // 원소 개수, 질의 개수
0 1 3
1 1 7
0 7 6
1 7 1
0 3 7
0 4 2
0 1 1
1 1 1
```

예제 출력 1

```
NO
NO
YES
```



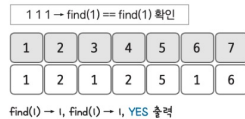
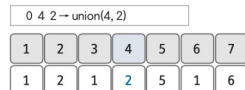
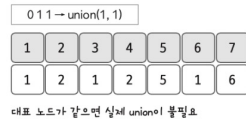
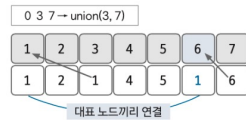
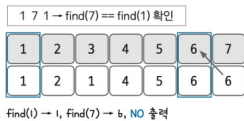
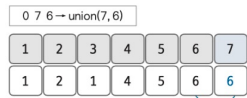
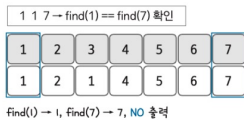
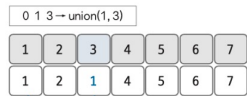
1 처음에는 노드가 연결돼 있지 않으므로 각 노드의 대표 노드는 자기 자신이다.

1	2	3	4	5	6	7
1	2	3	4	5	6	7

2 find 연산으로 특정 노드의 대표 노드를 찾는다.

union 연산으로 2개의 노드를 이용해 각 대표 노드를 찾아 연결한다.

질의한 값에 따라 결과를 반환한다.



Python

```
import sys
sys.setrecursionlimit(10**5)

n, m = map(int, sys.stdin.readline().rstrip().split())

parent = [i for i in range(n+1)]

def get_parent(x):
    if parent[x] == x:
        return x
    parent[x] = get_parent(parent[x]) # get_parent 거슬러 올라가면서 parent[x] 값도 갱신
    return parent[x]

def union_parent(a, b):
    a = get_parent(a)
    b = get_parent(b)

    if a < b: # 작은 쪽이 부모가 된다. (한 집합 관계라서 부모가 따로 있는 건 아님)
        parent[b] = a
    else:
        parent[a] = b

def find_parent(a, b):
    return get_parent(a) == get_parent(b)

for _ in range(m):
    op, a, b = map(int, sys.stdin.readline().rstrip().split())
    if op == 0:
        union_parent(a, b)
    else:
        print('YES' if find_parent(a, b) else 'NO')
```

Java, JS

집합 표현하기



Java

```
public class P003 {
    static int[] parent;

    static int getParent(int x) {
        if (parent[x] == x) {
            return x;
        }
        return parent[x] = getParent(parent[x]);
    }

    static void unionParent(int a, int b) {
        a = getParent(a);
        b = getParent(b);

        if (a < b) {
            parent[b] = a;
        } else {
            parent[a] = b;
        }
    }

    static boolean findParent(int a, int b) {
        // 두 원소의 루트 노드가 같은지 비교
        return getParent(a) == getParent(b);
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        StringBuilder sb = new StringBuilder();

        int n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());

        // 1. parent 배열 초기화 (각 노드의 부모를 자기 자신으로 설정)
        parent = new int[n + 1];
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
        }

        // 2. m개의 연산(Operation) 수행
        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int op = Integer.parseInt(st.nextToken());
            int a = Integer.parseInt(st.nextToken());
            int b = Integer.parseInt(st.nextToken());

            if (op == 0) {
                // 0번 연산: 합집합 (Union)
                unionParent(a, b);
            } else {
                // 1번 연산: 같은 집합인지 확인 (Find)
                sb.append(findParent(a, b) ? "YES" : "NO").append("\n");
            }
        }

        // 3. 결과 출력
        System.out.print(sb.toString());
    }
}
```

```
// 2. m개의 연산(Operation) 수행
for (int i = 0; i < m; i++) {
    st = new StringTokenizer(br.readLine());
    int op = Integer.parseInt(st.nextToken());
    int a = Integer.parseInt(st.nextToken());
    int b = Integer.parseInt(st.nextToken());

    if (op == 0) {
        // 0번 연산: 합집합 (Union)
        unionParent(a, b);
    } else {
        // 1번 연산: 같은 집합인지 확인 (Find)
        sb.append(findParent(a, b) ? "YES" : "NO").append("\n");
    }
}

// 3. 결과 출력
System.out.print(sb.toString());
}
```




JS

```
const readline = require("readline");
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});
let input = [];
rl.on("line", function (line) {
  input.push(line);
}).on("close", function () {
  solution(input);
  process.exit();
});

function solution(input) {
  const [N, M] = input.shift().split(" ").map(Number);
  const nodes = input.map((e) => e.split(" ").map(Number));
  const parents = new Array(N + 1).fill(-1);

  function find(x) {
    if (parents[x] < 0) return x;
    parents[x] = find(parents[x]);
    return parents[x];
  }

  function union(x, y) {
    x = find(x);
    y = find(y);
    if (x !== y) {
      parents[x] = y;
    }
  }

  const answer = new Array();
  nodes.forEach(([type, x, y]) => {
    if (type === 0) union(x, y);
    else answer.push(find(x) === find(y) ? "YES" : "NO");
  });

  console.log(answer.join("\n"));
}
```

Python

줄 세우기

**문제[053] 줄 세우기**

시간 제한 2초 | 난이도 골드 III | 백준 온라인 저지 2252번

N명의 학생들을 키 순서대로 줄을 세우려고 한다. 각 학생의 키를 직접 재서 정렬하면 간단하겠지만, 마땅한 방법이 없어 두 학생의 키를 비교하는 방법을 사용하기로 했다. 그나마도 모든 학생을 비교해 본 것이 아니라 일부 학생들의 키만을 비교해 봤다. 일부 학생들의 키를 비교한 결과가 주어졌을 때 줄을 세우는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 $N(1 \leq N \leq 32,000)$, $M(1 \leq M \leq 100,000)$ 이 주어진다. M은 키를 비교한 횟수다. 그다음 M개의 줄에는 키를 비교한 두 학생의 번호 A, B가 주어진다. 이는 학생 A가 학생 B의 앞에 서야 한다는 의미다. 학생들의 번호는 1번부터 N번이다.

↑ 출력

1번째 줄부터 앞에서부터 줄을 세운 결과를 출력한다. 답이 여러 가지일 경우에는 아무거나 출력한다.

예제 입력 1

```
4 2 // 노드 개수, 에지 개수
4 2
3 1
```

예제 출력 1

```
4 2 3 1
```

예제 입력 2

```
3 2
1 3
2 3
```

예제 출력 2

```
1 2 3
```

The diagram illustrates the process of converting a linked list into an adjacency list. It starts with a linked list where nodes 1 and 2 point to node 3, and nodes 2 and 4 point to node 4. This is then converted into an `ArrayList<Integer>[N]` array, where the first list contains [1, 2, 3] and the second list contains [2, 4]. Finally, this is converted into an adjacency list (진입 차수 배열) represented as a 2D array:

1	2	3	4
1	1	0	0

.

2 다음 순서에 따라 위상 정렬을 수행한다.

- ① 진입 차수가 0인 노드를 큐에 저장한다.
- ② 큐에서 데이터를 poll해 해당 노드를 탐색 결과에 추가하고, 해당 노드가 가리키는 노드의 진입 차수를 1씩 감소한다.
- ③ 감소했을 때 진입 차수가 0이 되는 노드를 큐에 offer한다.
- ④ 큐가 빌 때까지 ①~③을 반복한다.



Python

```
import sys
from collections import deque
input = sys.stdin.readline

N, M = map(int, input().split())
graph = [[] for _ in range(N+1)]
inDegree = [0]*(N+1)

for _ in range(M):
    A, B = map(int, input().split())
    graph[A].append(B)
    inDegree[B] += 1

q = deque()

for s in range(1, N+1):
    if inDegree[s] == 0:
        q.append(s)

ans = []

while q:
    s = q.popleft()
    ans.append(s)

    for adj_s in graph[s]:
        inDegree[adj_s] -= 1
        if inDegree[adj_s] == 0:
            q.append(adj_s)

print(*ans, sep=" ")
```

Java, JS

줄 세우기



Java

```

public class P004 {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int n = Integer.parseInt(st.nextToken()); // 노드(학생) 수
        int m = Integer.parseInt(st.nextToken()); // 간선(키 비교) 수

        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }
        int[] inDegree = new int[n + 1];

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int a = Integer.parseInt(st.nextToken());
            int b = Integer.parseInt(st.nextToken());

            // a가 b보다 앞에 서야 함 (a -> b)
            graph.get(a).add(b);
            // b의 진입 차수 증가
            inDegree[b]++;
        }
        Queue<Integer> queue = new LinkedList<>();
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i <= n; i++) {
            if (inDegree[i] == 0) {
                queue.add(i);
            }
        }
        while (!queue.isEmpty()) {
            int s = queue.poll();
            sb.append(s).append(" ");

            for (int adj_s : graph.get(s)) {
                inDegree[adj_s]--;

                if (inDegree[adj_s] == 0) {
                    queue.add(adj_s);
                }
            }
        }
        System.out.println(sb.toString().trim());
    }
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const [n, m] = input[0].split(' ').map(Number);
const graph = Array.from({ length: n + 1 }, () => []);
const inDegree = new Array(n + 1).fill(0);

for (let i = 1; i <= m; i++) {
    const [a, b] = input[i].split(' ').map(Number);
    // a -> b (a가 b의 선행)
    graph[a].push(b);
    // b의 진입 차수 증가
    inDegree[b]++;
}

const queue = [];
let head = 0; // shift()를 대체할 포인터
const ans = [];

for (let i = 1; i <= n; i++) {
    if (inDegree[i] === 0) {
        queue.push(i);
    }
}

while (head < queue.length) {
    // shift() 대신 head 포인터로 O(1) 디큐
    const s = queue[head++];
    // 결과 배열에 추가
    ans.push(s);

    // 현재 노드(s)와 연결된 노드들(adj_s)을 순회
    for (const adj_s of graph[s]) {
        // 연결된 노드의 진입 차수 1 감소
        inDegree[adj_s]--;

        // 진입 차수가 0이 되었다면, 큐에 추가
        if (inDegree[adj_s] === 0) {
            queue.push(adj_s);
        }
    }
}

console.log(ans.join(' '));

```

Python

게임 개발하기



빈출

문제[054] 게임 개발하기

시간 제한 2초 | 난이도 골드 III | 백준 온라인 저지 1516번

숨 회사에서 이번에 새로운 전략 시뮬레이션 게임 세준크래프트를 개발하기로 했다. 핵심적인 부분은 개발이 끝난 상태고, 종족별 균형과 전체 게임 시간 등을 조절하는 부분만 남아 있었다. 게임 플레이에 들어가는 시간은 상황에 따라 다를 수 있기 때문에 모든 건물을 짓는 데 걸리는 최소의 시간을 이용해 근사하기로 했다.

물론, 어떤 건물을 짓기 위해서는 다른 건물을 먼저 지어야 할 수도 있으므로 문제가 단순하지는 않다. 예를 들면 스타크래프트에서 병커를 짓기 위해서는 배럭을 먼저 지어야 하므로 배럭을 먼저 지은 후 병커를 지어야 한다. 여러 개의 건물을 동시에 지을 수 있다. 편의상 자원은 무한히 많고, 건물을 짓는 명령을 내리기까지는 시간이 걸리지 않는다고 가정해 보자. N개의 건물을 지을 때 각 건물을 짓기 위해 필요한 최소 시간을 출력하시오.

↓ 입력

1번째 줄에 건물의 종류 수 N ($1 \leq N \leq 500$), 그다음 N개의 줄에는 각 건물을 짓는 데 걸리는 시간과 그 건물을 짓기 위해 먼저 지어야 하는 건물들의 번호가 주어진다. 건물의 번호는 1부터 N까지로 하고, 각 줄은 -1로 끝난다고 가정해 보자. 각 건물을 짓는 데 걸리는 시간은 100,000보다 작거나 같은 자연수다.

↑ 출력

N개의 각 건물이 완성되기까지 걸리는 최소 시간을 출력한다.

예제 입력 1

```
5 // 건물 종류 수
10 -1
10 1 -1
4 1 -1
4 3 1 -1
3 3 -1
```

예제 출력 1

```
10
20
14
18
17
```

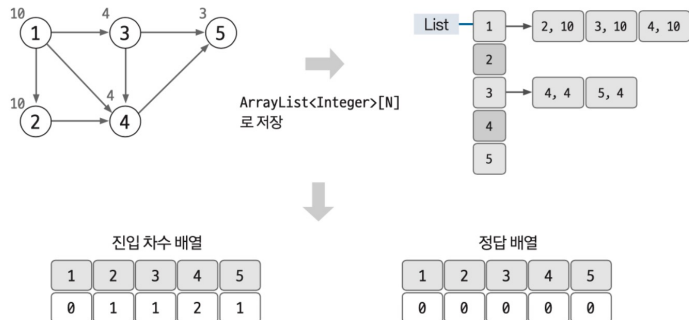



Python

게임 개발하기 - 정답

1 인접 리스트로 표현할 때 (인접 노드, 건물 번호)를 Node로 선언하여 연결한다.

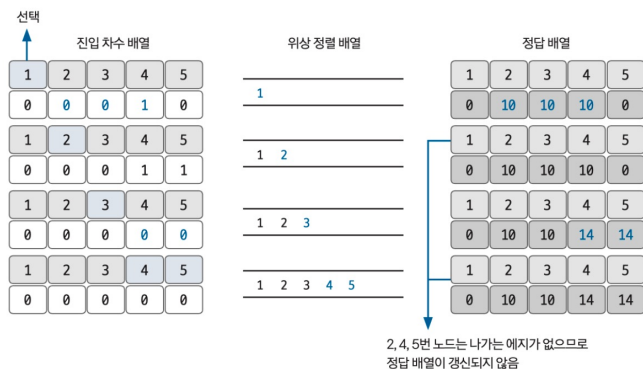
진입 차수 배열은 [0, 1, 1, 2, 1], 정답 배열은 모두 0으로 초기화한다.



2 위상 정렬을 실행하면서 각 건물을 짓는 데 걸리는 최대 시간을 업데이트 한다.

진입 차수 배열과 위상 정렬 배열, 정답 배열 업데이트 방법

$\text{Math.max}(\text{현재 건물(노드)에 저장된 최대 시간, 이전 건물(노드)에 저장된 최대 시간} + \text{현재 건물(노드)의 생산 시간})$



3 정답 배열에 자기 건물을 짓는 데 걸리는 시간을 더한 후 정답 배열을 차례대로 출력한다.



Python

```
import sys
from collections import deque

input = sys.stdin.readline
N = int(input())

building = [[] for _ in range(N + 1)]
indegree = [0] * (N + 1)
cost = [0] * (N + 1)

for i in range(1, N + 1):
    data = list(map(int, input().split()))[:-1]
    cost[i] = data[0]
    building_data = data[1:]

    # 간선 연결 및 진입차수 설정
    for j in building_data:
        building[j].append(i)
        indegree[i] += 1

def topology_sort():
    result = [0] * (N + 1)
    q = deque()

    for i in range(1, N + 1):
        if indegree[i] == 0:
            q.append(i)

    while q:
        now = q.popleft()
        # 큐에서 꺼낸 노드 번호에 해당하는 건물을 짓는데 걸리는 시간 저장
        # 선수 건물 짓는데 걸리는 시간이 포함되어 있음
        # 즉, '선수 건물 짓는데 걸리는 시간 + 현재 건물 짓는데 걸리는 시간'이 저장됨
        result[now] += cost[now]
        for b in building[now]:
            indegree[b] -= 1
            # b번호 건물을 짓기 전에 먼저 지어야 하는 선수 건물 짓는데 걸리는 시간으로 갱신
            result[b] = max(result[b], result[now])
            if indegree[b] == 0:
                q.append(b)

    return result

answer = topology_sort()
for i in range(1, N + 1):
    print(answer[i])
```

Java, JS

게임 개발하기



Java

```
public class P005 {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;

        int n = Integer.parseInt(br.readLine());

        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        int[] indegree = new int[n + 1];
        int[] cost = new int[n + 1];
        // result 배열은 '선수 건물이 모두 완료되는 데 걸리는 시간'을 저장
        int[] result = new int[n + 1];

        // 1. 입력 파싱
        for (int i = 1; i <= n; i++) {
            st = new StringTokenizer(br.readLine());

            cost[i] = Integer.parseInt(st.nextToken()); // 건물 짓는 비용(시간)

            while (st.hasMoreTokens()) {
                int preBuilding = Integer.parseInt(st.nextToken());
                if (preBuilding == -1) {
                    break;
                }
                // 간선 연결: preBuilding -> i
                graph.get(preBuilding).add(i);
                indegree[i]++; // i의 진입 차수 증가
            }
        }
    }
}
```

```
// 2. 위상 정렬 (Kahn's Algorithm)
Queue<Integer> queue = new LinkedList<>();

// 진입 차수가 0인 노드 큐에 추가
for (int i = 1; i <= n; i++) {
    if (indegree[i] == 0) {
        queue.add(i);
    }
}

while (!queue.isEmpty()) {
    int now = queue.poll();

    // * 현재 건물 완공 시간 = (선수 건물 완료 시간) + (현재 건물 짓는 시간)
    result[now] += cost[now];

    for (int b : graph.get(now)) {
        indegree[b]--;
        result[b] = Math.max(result[b], result[now]);

        if (indegree[b] == 0) {
            queue.add(b);
        }
    }
}

// 3. 결과 출력
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= n; i++) {
    sb.append(result[i]).append("\n");
}
System.out.print(sb.toString());
}
```



JS

게임 개발하기 - 정답

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const n = Number(input[0]);

// 2. 그래프 및 배열 초기화
const graph = Array.from({ length: n + 1 }, () => []);
const indegree = new Array(n + 1).fill(0);
const cost = new Array(n + 1).fill(0);
// result: 각 건물이 시작하기까지(선수 건물이 모두 완료되기까지) 걸리는 시간
const result = new Array(n + 1).fill(0);

// 3. 입력 파싱
for (let i = 1; i <= n; i++) {
    // 마지막 -1을 제외하고 파싱
    const data = input[i].split(' ').map(Number).slice(0, -1);

    cost[i] = data[0]; // 건물 짓는 시간
    const preBuildings = data.slice(1); // 선수 건물 목록

    for (const j of preBuildings) {
        // 간선 연결: j -> i
        graph[j].push(i);
        indegree[i]++; // i의 진입 차수 증가
    }
}

// 4. 위상 정렬 (Kahn's Algorithm)
const queue = [];
let head = 0;

// 진입 차수가 0인 노드 큐에 추가
for (let i = 1; i <= n; i++) {
    if (indegree[i] === 0) {
        queue.push(i);
    }
}

const answer = new Array(n + 1).fill(0); // 최종 완료 시간을 저장할 배열
```

```
while (head < queue.length) {
    const now = queue[head++];

    // * 현재 건물 완공 시간 = (선수 건물 완료 시간) + (현재 건물 짓는 시간)
    const completionTime = result[now] + cost[now];
    answer[now] = completionTime; // 최종 답안에 저장

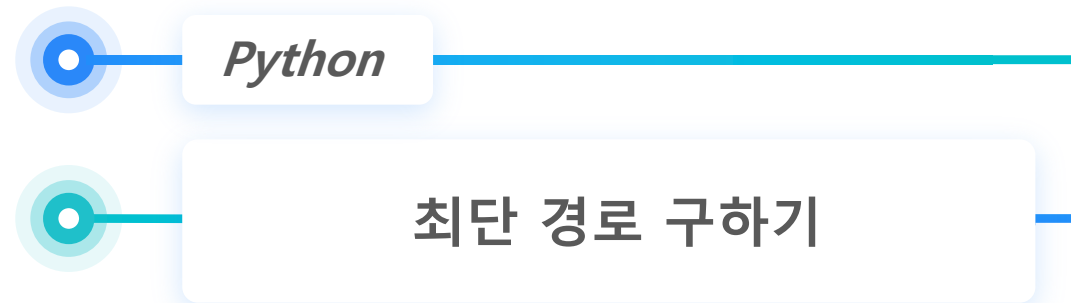
    for (const b of graph[now]) {
        indegree[b]--;

        // * b의 선수 건물 완료 시간 갱신
        // b의 선수 건물(now)의 완료 시간(completionTime)과
        // b의 다른 선수 건물들의 완료 시간(result[b]) 중 더 오래 걸리는 시간으로 갱신
        result[b] = Math.max(result[b], completionTime);

        if (indegree[b] === 0) {
            queue.push(b);
        }
    }
}

// 5. 결과 출력
const output = [];
for (let i = 1; i <= n; i++) {
    output.push(answer[i]);
}

console.log(output.join('\n'));
```





빈출

문제[056] 최단 경로 구하기

시간 제한 1초 | 난이도 골드 V | 백준 온라인 저지 1753번

에지의 가중치가 10 이하의 자연수인 방향 그래프가 있다. 이 그래프의 시작점에서 다른 모든 노드로의 최단 경로를 구하시오.

↓ 입력

1번째 줄에 노드의 개수 V 와 에지의 개수 E 가 주어진다($1 \leq V \leq 20,000, 1 \leq E \leq 300,000$). 모든 노드에는 1부터 V 까지 번호가 매겨져 있다. 2번째 줄에 출발 노드의 번호 K 가 주어진다($1 \leq K \leq V$). 3번째 줄에서 E 개의 줄에 걸쳐 각 에지의 정보(u, v, w)가 순서대로 주어진다. 이는 u 에서 v 로 가는 가중치 w 인 에지가 존재한다는 뜻으로, u 와 v 는 서로 다르다. 두 노드 사이에 에지가 2개 이상 존재할 수 있다는 것에 유의하자.

↑ 출력

1번째 줄부터 V 개의 줄에 걸쳐, i 번째 줄에 i 번 노드까지 최단 경로값을 출력한다. 시작점은 0, 경로가 없을 때는 INF를 출력한다.

예제 입력 1

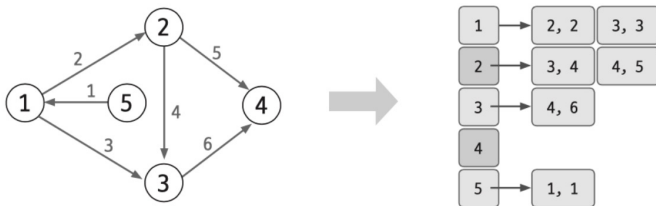
```
5 6 // 노드 개수, 에지 개수
1
5 1 1
1 2 2
1 3 3
2 3 4
2 4 5
3 4 6
```

예제 출력 1

```
0
2
3
7
INF
```



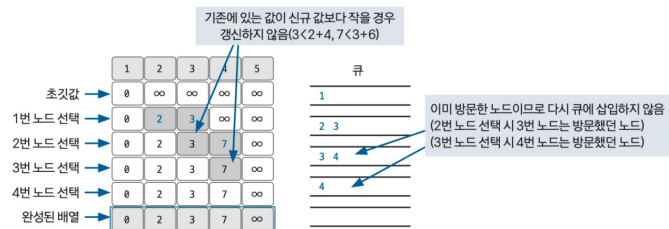
1 인접 리스트에 노드를 저장하고 거리 배열을 초기화한다.



2 최초 시작점을 큐에 삽입하고, 다음 과정에 따라 다익스트라 알고리즘을 수행한다.

다익스트라 알고리즘 수행 과정

- 거리 배열에서 아직 방문하지 않은 노드 중 현재 값이 가장 작은 노드를 선택한다.
- 해당 노드와 연결된 노드들의 최단 거릿값을 다음 공식을 이용해 업데이트한다.
 - Min(선택 노드의 거리 배열의 값 + 에지의 가중치, 연결 노드의 거리 배열의 값이 업데이트된 경우 연결 노드를 큐에 삽입)
- 큐가 빌 때까지 ①~②를 반복한다.



3 완성된 거리 배열의 값을 출력한다.



Python

```
import sys
import heapq

input = sys.stdin.readline
INF = int(1e9)

def dijkstra(start):
    q = []
    heapq.heappush(q, (0, start))
    distance[start] = 0

    while q:
        dist, now = heapq.heappop(q)

        if distance[now] < dist:
            continue

        # 현재 노드와 연결된 인접 노드 확인
        for i in graph[now]:
            cost = dist + i[1]
            if cost < distance[i[0]]:
                distance[i[0]] = cost
                heapq.heappush(q, (cost, i[0]))

# V == 5일 때 1~5까지 노드가 있는거임.
V, E = map(int, input().split())

snode = int(input()) # 시작 노드

graph = [[] for _ in range(V+1)]
distance = [INF] * (V+1) # 최단 거리 테이블
# 연결 정보 입력
for _ in range(E):
    u, v, w = map(int, input().split())
    graph[u].append((v, w))

dijkstra(snode)

# i번째 줄에 i번 정점으로의 최단 경로의 경로값을 출력
for i in range(1, V+1):
    if distance[i] == INF:
        print("INF")
    else:
        print(distance[i])
```

Java, JS

최단 경로 구하기



Java

```

public class P006 {

    static class Node implements Comparable<Node> {
        int index; // 노드 번호
        int cost; // 해당 노드까지의 거리 (가중치)

        public Node(int index, int cost) {
            this.index = index;
            this.cost = cost;
        }

        // 거리를 기준으로 오름차순 정렬
        @Override
        public int compareTo(Node other) {
            // int 비교는 단순 뺄셈으로 가능
            return this.cost - other.cost;
        }
    }

    static final int INF = (int) 1e9; // 무한대 값
    static ArrayList<ArrayList<Node>> graph;
    static int[] distance;
    static int v, e, snode; // v: 정점 수, e: 간선 수, snode: 시작 노드

    static void dijkstra(int start) {
        PriorityQueue<Node> pq = new PriorityQueue<>();

        // 1. 시작 노드 설정
        pq.add(new Node(start, 0));
        distance[start] = 0;

        while (!pq.isEmpty()) {
            Node node = pq.poll();
            int dist = node.cost; // 현재 노드까지의 거리
            int now = node.index; // 현재 노드 번호

            // 2. 이미 처리된 노드(더 짧은 경로를 찾은 경우)는 무시
            if (distance[now] < dist) {
                continue;
            }
        }
    }
}

```

```

        // 3. 현재 노드와 연결된 인접 노드 확인
        for (Node neighbor : graph.get(now)) {
            // neighbor.index = 인접 노드 번호
            // neighbor.cost = now에서 neighbor까지의 가중치
            int cost = dist + neighbor.cost; // 현재 노드를 거쳐 가는 비용

            // 4. 현재 노드를 거쳐 가는 것이 더 짧은 경우
            if (cost < distance[neighbor.index]) {
                distance[neighbor.index] = cost;
                // 우선순위 큐에 (갱신된 거리, 인접 노드) 추가
                pq.add(new Node(neighbor.index, cost));
            }
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        v = Integer.parseInt(st.nextToken());
        e = Integer.parseInt(st.nextToken());
        snode = Integer.parseInt(br.readLine());

        // 1. 그래프 및 최단 거리 테이블 초기화
        graph = new ArrayList<>();
        for (int i = 0; i <= v; i++) {
            graph.add(new ArrayList<>());
        }

        distance = new int[v + 1];
        Arrays.fill(distance, INF);
    }
}

```

```

        // 2. 간선 정보 입력
        for (int i = 0; i < e; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken()); // 출발
            int v_node = Integer.parseInt(st.nextToken()); // 도착
            int w = Integer.parseInt(st.nextToken()); // 가중치
            graph.get(u).add(new Node(v_node, w));
        }

        // 3. 다익스트라 알고리즘 수행
        dijkstra(snode);

        // 4. 결과 출력
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i <= v; i++) {
            if (distance[i] == INF) {
                sb.append("INF").append("\n");
            } else {
                sb.append(distance[i]).append("\n");
            }
        }
        System.out.print(sb.toString());
    }
}

```



JS

```
// 1. 최소 힙(Min Heap) 구현
// (JavaScript에는 표준 우선순위 큐가 없으므로 직접 구현)
class MinPriorityQueue {
  constructor() {
    this.heap = [];
  }

  isEmpty() {
    return this.heap.length === 0;
  }

  // 삽입 (O(log N))
  push(value) { // [거리, 노드]
    this.heap.push(value);
    this.bubbleUp();
  }

  // 삭제 (O(log N))
  pop() {
    if (this.isEmpty()) return null;
    if (this.heap.length === 1) return this.heap.pop();

    const min = this.heap[0];
    this.heap[0] = this.heap.pop();
    this.bubbleDown();
    return min;
  }

  // --- 내부 헬퍼 함수 ---
  bubbleUp() {
    let index = this.heap.length - 1;
    const lastNode = this.heap[index];

    while (index > 0) {
      const parentIndex = Math.floor((index - 1) / 2);
      const parentNode = this.heap[parentIndex];

      // 부모의 거리(cost)가 더 크면 swap
      if (parentNode[0] > lastNode[0]) {
        this.heap[index] = parentNode;
        index = parentIndex;
      } else {
        break;
      }
    }
  }
}
```

```
    this.heap[index] = lastNode;
  }

  bubbleDown() {
    let index = 0;
    const rootNode = this.heap[index];
    const len = this.heap.length;

    while (true) {
      let leftChildIndex = index * 2 + 1;
      let rightChildIndex = index * 2 + 2;
      let swapIndex = -1;

      if (leftChildIndex < len) {
        if (this.heap[leftChildIndex][0] < rootNode[0]) {
          swapIndex = leftChildIndex;
        }
      }

      if (rightChildIndex < len) {
        if (
          (swapIndex === -1 && this.heap[rightChildIndex][0] < rootNode[0]) ||
          (swapIndex !== -1 && this.heap[rightChildIndex][0] < this.heap[leftChildIndex][0])
        ) {
          swapIndex = rightChildIndex;
        }
      }

      if (swapIndex === -1) break;

      this.heap[index] = this.heap[swapIndex];
      index = swapIndex;
    }
    this.heap[index] = rootNode;
  }
}
```

```
// 2. 입력 처리
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const [v, e] = input[0].split(' ').map(Number);
const snode = Number(input[1]);
const INF = 1e9; // 무한대 값

// 3. 그래프 및 최단 거리 테이블 초기화
const graph = Array.from({ length: v + 1 }, () => []);
const distance = new Array(v + 1).fill(INF);

// 4. 간선 정보 입력
for (let i = 2; i < e + 2; i++) {
  const [u, v, w] = input[i].split(' ').map(Number);
  // graph[u]에 [목적노드, 가중치] 저장
  graph[u].push([v, w]);
}

// 5. 다익스트라 함수
function dijkstra(start) {
  const pq = new MinPriorityQueue();

  // 1. 시작 노드 설정
  pq.push([0, start]); // [거리, 노드번호]
  distance[start] = 0;

  while (!pq.isEmpty()) {
    const [dist, now] = pq.pop();

    // 2. 이미 처리된 노드(더 짧은 경로를 찾은 경우)는 무시
    if (distance[now] < dist) {
      continue;
    }

    // 3. 현재 노드와 연결된 인접 노드 확인
    for (const [neighbor, weight] of graph[now]) {
      const cost = dist + weight; // 현재 노드를 거쳐 가는 비용

      // 4. 현재 노드를 거쳐 가는 것이 더 짧은 경우
      if (cost < distance[neighbor]) {
        distance[neighbor] = cost;
        pq.push([cost, neighbor]);
      }
    }
  }
}

// 6. 다익스트라 알고리즘 수행
dijkstra(snode);

// 7. 결과 출력
const output = [];
for (let i = 1; i <= v; i++) {
  if (distance[i] === INF) {
    output.push("INF");
  } else {
    output.push(distance[i]);
  }
}
console.log(output.join('\n'));
```

Python

K 번째 최단 경로 찾기



핵심

문제[058] K번째 최단 경로 찾기

시간 제한 2초 | 난이도 플래티넘 | 백준 온라인 저지 1854번

봄 캠프를 마친 김 조교는 여러 도시를 돌며 여행을 다닐 계획이다. 그런데 김 조교는 ‘느림의 미학’을 중요시하는 사람이라 항상 최단 경로로만 이동하는 것은 별로 좋아하지 않는다. 하지만 너무 시간이 오래 걸리는 경로도 그리 매력적인 것만은 아니어서 적당한 타협안인 ‘K번째 최단 경로’를 구하길 원한다. 그를 돕기 위한 프로그램을 작성해 보자.

↓ 입력

1번째 줄에 n, m, k 가 주어진다. ($1 \leq n \leq 1000, 0 \leq m \leq 2000000, 1 \leq k \leq 100$) n 과 m 은 각각 김 조교가 여행을 고려하고 있는 도시들의 개수와 도시 간에 존재하는 도로의 수다. 이어지는 m 개의 줄에는 각각 도로의 정보를 제공하는 3개의 정수 a, b, c 가 포함돼 있다. 이것은 a 번 도시에서 b 번 도시로 갈 때는 c 의 시간이 걸린다는 의미다($1 \leq a, b \leq n, 1 \leq c \leq 1000$). 도시의 번호는 1번부터 n 번까지 연속해 있고, 1번은 시작 도시다.

↑ 출력

n 개의 줄을 출력한다. i 번째 줄에 1번 도시에서 i 번 도시로 가는 K번째 최단 경로의 소요 시간을 출력한다. 경로의 소요 시간은 경로 위에 있는 도로들을 따라 이동하는 데 필요한 시간들의 합이다. i 번 도시에서 i 번 도시로 가는 최단 경로는 0이지만, 일반적인 K번째 최단 경로는 0이 아닐 수 있다는 것에 유의한다. 또 K번째 최단 경로가 존재하지 않으면 -1을 출력한다. 최단 경로에 같은 노드가 여러 번 포함될 수 있다.

예제 입력 1

```
5 10 2 // 도시 수, 도로 수, K
1 2 2
1 3 7
1 4 5
1 5 6
2 4 2
2 3 4
3 4 6
3 5 8
5 2 4
5 4 1
```

예제 출력 1

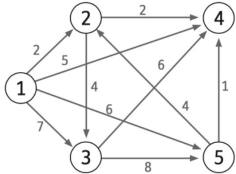
```
-1
10
7
5
14
```



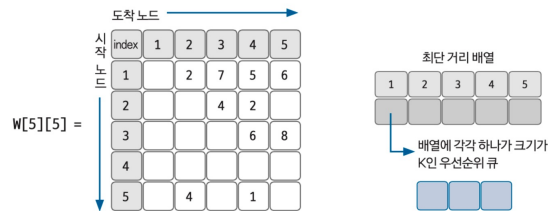
Python

K 번째 최단 경로 찾기 - 정답

- 주어진 예제 데이터를 기반으로 그래프를 그린다.
도시는 노드로, 도로는 에지로 나타낸다.



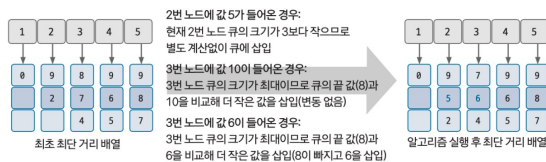
- 변수를 선언하고, 그래프 데이터를 받는 부분은 모두 다익스트라 알고리즘 준비 과정과 동일하다.



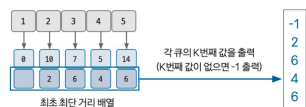
- 최단 거리 배열을 우선순위 큐 배열로 선언하고, 다음과 같은 기준을 세워 채운다.
예제에서 주어진 입력에서 K=2이지만, 이해를 돕기 위해 K=3일 때를 계산해보았다.

최단 거리 배열 채우기 규칙

- 현재 노드에 저장된 경로가 K개 미만일 때 신규 경로를 추가한다.
- 경로가 K개일 때 현재 경로 중 최대 경로와 신규 경로를 비교해 신규 경로가 더 작을 때 업데이트한다. 우선 순위 큐를 사용하면 이 로직을 쉽게 구현할 수 있다.
- K번째 경로를 찾기 위해서는 노드를 여러 번 쓰는 경우가 생기므로 사용한 노드는 방문 배열에 확인해 두고 재사용하지 않는 부분은 삭제한다.



- 최단 거리 배열을 탐색하면서 K번째 경로가 존재하면 출력하고, 존재하지 않으면 -1을 출력한다.



Python

```
import sys
import heapq

input = sys.stdin.readline
INF = int(1e12)

# 1 ≤ n ≤ 1000, 0 ≤ m ≤ 2000000, 1 ≤ k ≤ 100
N, M, K = map(int, input().split())

# 거리, 위치
q = []
# a번 도시에서 b번 도시로 갈 때는 c의 시간이 걸린다. (1 ≤ a, b ≤ n. 1 ≤ c ≤ 1000)
distance = [[] for _ in range(N+1)]
graph = [[] for _ in range(N+1)]

for _ in range(M):
    a, b, c = map(int, input().split())
    graph[a].append((c, b))

heapq.heappush(distance[1], 0) # 비용
heapq.heappush(q, (0, 1)) # 비용, 장소

while q:
    dist, now = heapq.heappop(q)
    for i in graph[now]:
        # i[0]: 비용, i[1]: 장소
        cost = dist + i[0]
        # 저장된 경로가 K보다 작을 때
        if len(distance[i[1]]) < K:
            # 그냥 넣기
            heapq.heappush(distance[i[1]], -cost)
            heapq.heappush(q, (cost, i[1]))
        # 저장된 경로가 K보다 크고
        # heapq의 가장 작은 값에 -1을 곱한 값보다 cost가 작을 때
        elif cost < -distance[i[1]][0]:
            # heapq에서 데이터 하나를 뺌
            heapq.heappop(distance[i[1]])
            heapq.heappush(distance[i[1]], -cost)
            heapq.heappush(q, (cost, i[1]))

for i in range(1, N+1):
    if len(distance[i]) == K:
        print(-distance[i][0])
    else:
        print(-1)
```

Java, JS

K 번째 최단 경로 찾기



Java

```
public class P007 {
    static class Node implements Comparable<Node> {
        int index;
        long cost;

        public Node(int index, long cost) {
            this.index = index;
            this.cost = cost;
        }

        @Override
        public int compareTo(Node other) {
            // long 타입 비교는 Long.compare 사용 권장
            return Long.compare(this.cost, other.cost);
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());
        int k = Integer.parseInt(st.nextToken());

        // graph[i] = (비용, 도착노드) -> (Node(도착노드, 비용))
        ArrayList<ArrayList<Node>> graph = new ArrayList<>();

        // distance[i] = i번 노드까지의 K개의 최단 경로를 저장하는 힙
        // (음수 비용을 저장하는 최소 힙 -> 양수 비용 기준 최대 힙)
        PriorityQueue<Long>[] distance = new PriorityQueue[n + 1];

        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
            distance[i] = new PriorityQueue<Long>();
        }

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int a = Integer.parseInt(st.nextToken());
            int b = Integer.parseInt(st.nextToken());
            int c = Integer.parseInt(st.nextToken());
            graph.get(a).add(new Node(b, c)); // (도착노드, 비용)
        }
    }
}
```

```
// 다익스트라 알고리즘 큐 (전체 경로 탐색용)
PriorityQueue<Node> q = new PriorityQueue<>();

// 1. 시작 노드 설정
q.add(new Node(1, 0L));
distance[1].add(0L); // 음수 0은 0이므로 0L

while (!q.isEmpty()) {
    Node node = q.poll();
    long dist = node.cost;
    int now = node.index;

    // 2. 현재 노드(now)와 연결된 인접 노드(neighbor) 확인
    for (Node neighbor : graph.get(now)) {
        // neighbor.index = 도착노드(b), neighbor.cost = 비용(c)
        long cost = dist + neighbor.cost;
        int nextNode = neighbor.index;

        // 3. K번째 경로 갱신
        // 3-1. 아직 K개의 경로가 채워지지 않았다면
        if (distance[nextNode].size() < k) {
            distance[nextNode].add(-cost); // (음수로 저장)
            q.add(new Node(nextNode, cost));
        }
        // 3-2. K개가 찼고, 새 경로(cost)가 K번째 경로(-distance[nextNode].peek())보다 작다면
        else if (cost < -distance[nextNode].peek()) {
            distance[nextNode].poll(); // K번째 경로(가장 큰 값) 삭제
            distance[nextNode].add(-cost); // 새 경로 추가 (음수로)
            q.add(new Node(nextNode, cost));
        }
    }
}

// 4. 결과 출력
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= n; i++) {
    if (distance[i].size() == k) {
        // K번째 경로는 최소 힙의 루트(음수)를 뒤집은 값
        sb.append(-distance[i].peek()).append("\n");
    } else {
        sb.append(-1).append("\n");
    }
}
System.out.print(sb.toString());
}
```



JS

```
// 1. 최소 힙(Min Heap) 구현
// (BigInt 비교가 가능하도록 수정)
class MinPriorityQueue {
  constructor() {
    this.heap = [];
  }
  isEmpty() {
    return this.heap.length === 0;
  }
  size() {
    return this.heap.length;
  }
  // [0]번째 요소를 기준으로 비교 (BigInt)
  push(value) {
    this.heap.push(value);
    this.bubbleUp();
  }
  pop() {
    if (this.isEmpty()) return null;
    if (this.heap.length === 1) return this.heap.pop();
    const min = this.heap[0];
    this.heap[0] = this.heap.pop();
    this.bubbleDown();
    return min;
  }
  peek() {
    return this.heap[0] || null;
  }
  // --- 내부 헬퍼 함수 ---
  bubbleUp() {
    let index = this.heap.length - 1;
    const lastNode = this.heap[index];
    while (index > 0) {
      const parentIndex = Math.floor((index - 1) / 2);
      const parentNode = this.heap[parentIndex];
      if (parentNode[0] > lastNode[0]) { // BigInt 비교
        this.heap[index] = parentNode;
        index = parentIndex;
      } else {
        break;
      }
    }
    this.heap[index] = lastNode;
  }
}
```

```
bubbleDown() {
  let index = 0;
  const rootNode = this.heap[index];
  const len = this.heap.length;
  while (true) {
    let leftChildIndex = index * 2 + 1;
    let rightChildIndex = index * 2 + 2;
    let swapIndex = -1;
    if (leftChildIndex < len) {
      if (this.heap[leftChildIndex][0] < rootNode[0]) {
        swapIndex = leftChildIndex;
      }
    }
    if (rightChildIndex < len) {
      if (
        (swapIndex === -1 && this.heap[rightChildIndex][0] < rootNode[0]) ||
        (swapIndex !== -1 && this.heap[rightChildIndex][0] < this.heap[leftChildIndex][0])
      ) {
        swapIndex = rightChildIndex;
      }
    }
    if (swapIndex === -1) break;
    this.heap[index] = this.heap[swapIndex];
    index = swapIndex;
  }
  this.heap[index] = rootNode;
}
```

```
// 2. 입력 처리
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');
let line = 0;

const [n, m, k] = input[line++].split(' ').map(Number);

// 3. 그래프 및 힙 배열 초기화
const graph = Array.from({ length: n + 1 }, () => []);
// distance[i] = i번 노드까지의 K개 최단 경로 (최대 힙 사용)
const distance = Array.from({ length: n + 1 }, () => new MinPriorityQueue());

// 4. 간선 정보 입력 (BigInt 사용)
for (let i = 0; i < m; i++) {
  const [a, b, c] = input[line++].split(' ').map(Number);
  // 비용(BigInt), 도착노드(Number)
  graph[a].push([BigInt(c), b]);
}

// 5. 다익스트라 큐 초기화
const q = new MinPriorityQueue(); // (현재 경로 합계)

// 6. 시작 노드 설정
// [비용(BigInt), 노드(Number)]
q.push([0n, 1]);
// distance 힙에는 [음수비용(BigInt)]만 저장
distance[1].push([0n]);

// 7. 다익스트라 수행
while (!q.isEmpty()) {
  const [dist, now] = q.pop(); // dist: BigInt, now: Number

  // 8. 인접 노드 탐색
  for (const [c, b] of graph[now]) { // c: BigInt, b: Number
    const cost = dist + c; // cost: BigInt
    const nextNode = b;

    // 9. K번째 경로 갱신
    // 9-1. 이미 K개의 경로가 채워지지 않았다면
    if (distance[nextNode].size() < k) {
      distance[nextNode].push([-cost]); // (음수 비용 저장)
      q.push([cost, nextNode]);
    }
  }
}
```

```
// 9-2. K개가 찼고, 새 경로(cost)가 K번째 경로(-distance[nextNode].peek()[0])보다 작다면
else if (cost < -distance[nextNode].peek()[0]) {
  distance[nextNode].pop();
  distance[nextNode].push([-cost]);
  q.push([cost, nextNode]);
}

// 10. 결과 출력
const output = [];
for (let i = 1; i <= n; i++) {
  if (distance[i].size() === k) {
    // K번째 경로는 최소 힙의 루트(음수)를 뒤집은 값
    // BigInt는 String()으로 변환하여 출력
    output.push(String(-distance[i].peek()[0]));
  } else {
    output.push("-1");
  }
}
console.log(output.join('\n'));
```


감사합니다!